03_advanced_pyspark_analytics

September 23, 2026

[1]:
```
# ============================================================================
# Für Spark Teil 1: Java 17 (erforderlich für PySpark 4.x) automatisch laden
# ============================================================================_
!python.exe -m pip install pyspark install-jdk

import os
import jdk
from pathlib import Path

print("Prüfe lokale Java 17 Installation...")
# Java 17 fest
JAVA_17_PATH = str(Path(os.path.expanduser("~")) / ".jdk" / "jdk-17.0.20.1+1")

# Falls Java 17 noch nicht existiert, lokales Zip/Paket laden
if not Path(JAVA_17_PATH).exists():
    print("Lade kompatibles Java 17 für PySpark herunter (ca. 50MB)...")
    jdk.install('17')

# Sucht den echten, frisch installierten Java 17 Ordner auf der Festplatte
base_jdk_path = Path(os.path.expanduser("~")) / ".jdk"
java_found_path = None
for item in base_jdk_path.iterdir():
    # Stellt sicher, dass es eine Version 17 gibt.
    if item.is_dir() and "jdk-17" in item.name and (item / "bin").exists():
        java_found_path = str(item)
        break

if java_found_path:
    os.environ["JAVA_HOME"] = java_found_path
    os.environ["PATH"] = java_found_path + "\\bin;" + os.environ.get("PATH", "")
    print("=" * 70)
    print("ERFOLG: Passendes Java 17 wurde im User-Ordner aktiviert!")
    print(f"Pfad registriert: {os.environ['JAVA_HOME']}")
    print(f"Überprüfung (Muss True sein): {os.path.exists(os.
 ↪environ['JAVA_HOME'])}")
    print("=" * 70)
else:
```

```
    print("FEHLER: Java 17 Ordner wurde nicht gefunden.")
```

```
Requirement already satisfied: pyspark in C:\0_DA\Python312\Lib\site-packages
(4.2.0)
Requirement already satisfied: install-jdk in C:\0_DA\Python312\Lib\site-
packages (1.1.0)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in
C:\0_DA\Python312\Lib\site-packages (from pyspark) (0.10.9.9)
Prüfe lokale Java 17 Installation…
======================================================================
ERFOLG: Passendes Java 17 wurde im User-Ordner aktiviert!
Pfad registriert: C:\Users\alfa\.jdk\jdk-17.0.20.1+1
Überprüfung (Muss True sein): True
======================================================================
```

1 Erweiterte Big-Data-Analyse: Untersuchung netzweiter Anschlusskonflikte mittels Apache PySpark

Dieses dedizierte Hilfsnotebook beinhaltet die Durchführung der rechenintensiven, erweiterten Analysen im Rahmen der Phase 2B. Während die makroskopische explorative Datenanalyse (EDA) im Hauptnotebook mittels Pandas die Verteilungscharakteristika und qualitative Datenmängel offengelegt hat, stößt die relationale Verarbeitung von komplexen, zeilenübergreifenden Abhängigkeiten bei über zwei Millionen Datensätzen an lokale Arbeitsspeicher- und Performancegrenzen.

1.1 Zielsetzung der erweiterten Analyse

Die Kernaufgabe dieses Abschnitts besteht darin, potenzielle Anschlussrisiken und systemische Kettenreaktionen im Schienennetz mathematisch zu identifizieren. Hierzu werden die zeitlichen Abfolgen von ankommenden und abfahrenden Zugläufen an identischen Netzknotenpunkten kombinatorisch miteinander verknüpft. Um diese komplexen Datenoperationen (Self-Joins und temporale Fensterfunktionen) performant und ohne Speicherüberlauf auszunehmen, wird das Big-Data-Framework Apache PySpark eingesetzt.

Die Ergebnisse dieser Berechnungen werden anschließend als hochverdichtete Aggregattabellen im spaltenorientierten Parquet-Format exportiert, um eine nahtlose Rückführung in das Hauptnotebook und die Integration in das finale interaktive Dashboard zu gewährleisten.

> *Hinweis zur Reproduzierbarkeit:* Sollte dieses Notebook auf einer anderen Systemumgebung ausgeführt werden, stellt die automatisierte Initialisierungszelle die Verfügbarkeit der Java-Laufzeitumgebung sowie der erforderlichen Python-Pakete sicher.
>
> Bereinigtes Datensatz aus der Phase 2 soll im Projekt-Notebook auch bereits produziert werden. Also Ausführung des `Projekt_DeutscheBahn.ipynb` bis zum `Explorative Datenanalyse und qualitative Evaluierung des Schienenverkehrs` vorzunehmen.

1.2 Laden der Projekt-Infrastruktur und Imports

Nach erfolgreicher Bereitstellung der Java-Laufzeitumgebung werden die benötigten Python-Bibliotheken, Spark-SQL-Funktionen sowie die zentralen Pfaddefinitionen aus der Projektkonfiguration geladen. Die Adressierung der physischen Dateien auf der Festplatte erfolgt konsequent über relative, POSIX-kompatible Pfade, um Kompatibilitätskonflikte zwischen der Python-Laufzeitumgebung und der Java Virtual Machine (JVM) systemisch auszuschließen.

[2]:
```
import sys
import time
import os
from pathlib import Path

# PySpark SQL Kernkomponenten laden
from pyspark.sql import SparkSession
from pyspark.sql.window import Window
import pyspark.sql.functions as F
from pyspark.sql.types import *
import pandas as pd
import shutil
from pyspark.storagelevel import StorageLevel

# Laden der zentralen Projektkonfiguration und der relativen Pfadmethode
from project_config import (
    CLEANED_DS_PARQUET_PATH,
    PYSPARK_RESULTS,
    get_project_relative_path
)

# Konvertierung des bereinigten Datensatz-Pfades in ein POSIX-kompatibles␣
 ↪relatives Format
CLEANED_DS_RELATIVE_STRING = get_project_relative_path(CLEANED_DS_PARQUET_PATH)

print(f"Zentraler bereinigter Datensatz (relativ):␣
 ↪{CLEANED_DS_RELATIVE_STRING}")
```

```
Zentraler bereinigter Datensatz (relativ): data/DBtrainrides_cleaned.parquet
```

1.3 Konfiguration und Initialisierung der SparkSession

Für die Durchführung der Big-Data-Analyse wird eine lokale SparkSession instanziiert. Die Konfiguration wird so optimiert, dass das Framework sämtliche verfügbaren CPU-Kerne parallel nutzt. Temporäre Verzeichnisse für das Scratch- und Warehouse-System werden explizit im Benutzerverzeichnis definiert, um Berechtigungskonflikte zu vermeiden. Zudem wird eine strikte UTF-8-Codierung erzwungen, um Sonderzeichen in den Bahnhofsnamen verlustfrei zu verarbeiten.

[3]:
```
# 1. Temporäre Verzeichnisse im Benutzerordner definieren
user_home = os.path.expanduser("~")
spark_warehouse = os.path.join(user_home, "spark-warehouse")
```

```
spark_scratch = os.path.join(user_home, "spark-scratch")

# Verzeichnisse sicherstellen
os.makedirs(spark_warehouse, exist_ok=True)
os.makedirs(spark_scratch, exist_ok=True)

print("Initialisiere lokale SparkSession...")
start_spark_init = time.perf_counter()

# 2. Aufbau der SparkSession mit Leistungsoptimierungen
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("AdvancedPySparkAnalytics")
    .config("spark.sql.warehouse.dir", str(spark_warehouse))
    .config("spark.local.dir", str(spark_scratch))
    .config("spark.driver.host", "127.0.0.1")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.ui.port", "4050")
    .config(
        "spark.driver.extraJavaOptions",
        "-Djava.net.preferIPv4Stack=true "
        "-Dfile.encoding=UTF-8 "
        "-Dsun.jnu.encoding=UTF-8"
    )
    .config(
        "spark.executor.extraJavaOptions",
        "-Dfile.encoding=UTF-8 "
        "-Dsun.jnu.encoding=UTF-8"
    )
    .getOrCreate()
)

spark_init_zeit = time.perf_counter() - start_spark_init

print(f"SparkSession erfolgreich gestartet.")
print(f"Initialisierungszeit: {spark_init_zeit:.2f} Sekunden")
```

```
Initialisiere lokale SparkSession…
```

```
C:\0_DA\Python312\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning:
PySpark does not yet fully support pandas >= 3.0.0. Some features may not work
correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
```

```
SparkSession erfolgreich gestartet.
Initialisierungszeit: 6.37 Sekunden
```

1.4 Daten-Ingestion und Schema-Verifikation

Nach erfolgreichem Start der Engine wird der zuvor in Phase 2 bereinigte und als Parquet-Datei gesicherte Datensatz in die Spark-Umgebung eingelesen. Da das Parquet-Format ein explizites Schema besitzt, entfällt ein rechenintensiver Typ-Inferenz-Prozess. Nach dem Laden wird das Schema ausgegeben, um die exakte Übereinstimmung der Datentypen (insbesondere der in Ganzzahlen zerlegten ID-Segmente und der temporalen Datetime-Objekte) zu verifizieren.

[4]:
```
print(f"Lese bereinigten Datensatz ein: {CLEANED_DS_RELATIVE_STRING}")

# Laden des DataFrames aus der relativen Pfad-Zeichenkette
df_spark = spark.read.parquet(CLEANED_DS_RELATIVE_STRING)

# Registrierung als temporäre SQL-View für flexible Abfragen
df_spark.createOrReplaceTempView("zug_daten")

# Strukturanalyse des Schemas
print("\n--- Strukturelles DataFrame-Schema ---")
df_spark.printSchema()

# Zählen der Datensätze zur Verifikation der Datenintegrität (Lazy Evaluation␣
 ↪auslösen)
start_count = time.perf_counter()
gesamt_zeilen = df_spark.count()
count_zeit = time.perf_counter() - start_count

print(f"\nErfolgreich geladen: {gesamt_zeilen:,} Zeilen.")
print(f"Zeit für den initialen Count-Check: {count_zeit:.2f} Sekunden")
```

```
Lese bereinigten Datensatz ein: data/DBtrainrides_cleaned.parquet

--- Strukturelles DataFrame-Schema ---
root
 |-- line: string (nullable = true)
 |-- path: string (nullable = true)
 |-- eva_nr: long (nullable = true)
 |-- category: long (nullable = true)
 |-- station: string (nullable = true)
 |-- state: string (nullable = true)
 |-- city: string (nullable = true)
 |-- zip: long (nullable = true)
 |-- long: double (nullable = true)
 |-- lat: double (nullable = true)
 |-- arrival_delay_m: long (nullable = true)
 |-- departure_delay_m: long (nullable = true)
 |-- info: string (nullable = true)
 |-- ID_part1: long (nullable = true)
 |-- ID_part2: long (nullable = true)
```

```
 |-- ID_part3: long (nullable = true)
 |-- arrival_plan: timestamp_ntz (nullable = true)
 |-- departure_plan: timestamp_ntz (nullable = true)
 |-- arrival_change: timestamp_ntz (nullable = true)
 |-- departure_change: timestamp_ntz (nullable = true)
 |-- departure_weekday: string (nullable = true)
 |-- is_arrival_delayed: boolean (nullable = true)
 |-- is_departure_delayed: boolean (nullable = true)


Erfolgreich geladen: 2,061,357 Zeilen.
Zeit für den initialen Count-Check: 1.34 Sekunden
```

1.5 Methodische Konzeption der kaskadierten Anschluss- und Kausalitätsanalyse

Die erweiterte Analyse wird in eine mehrstufige relationale Pipeline unterteilt, um sowohl direkte Anschlussverluste als auch netzweite Verspätungsübertragungen (Sekundärverspätungen) isoliert zu erforschen. Das Verarbeitungsmodell basiert auf der sequenziellen Erstellung temporärer Datenbankansichten (TempViews) in PySpark:

1. **Etablierung der Anschluss-Baseline:** Generierung einer primären Ansicht aller fahrplanmäßig plausiblen Umstiegsverbindungen innerhalb eines erweiterten Korridors von **2 bis 50 Minuten** an identischen Betriebszuständen (`eva_nr`).
2. **Identifikation von Anschlussverlusten:** Mathematische Filterung jener Verbindungen, bei denen die tatsächliche Ankunftszeit des Zubringers die geplante Abfahrtszeit des Anschlusses überschreitet.
3. **Analyse von Folgeverspätungen (Wartezeit-Effekt):** Untersuchung von Ereignissen, bei denen ein Anschlusszug eine tatsächliche Abfahrtsverzögerung aufweist, die zeitlich mit der verspäteten Ankunft eines verknüpften Zubringers korreliert. Dies dient als Indikator für betriebliche Wartezeiten zur Sicherung von Umstiegsströmen.
4. **Kausalitätsprüfung über den Haltestellen-Index:** Einbeziehung des sequenziellen Fahrt-Indikators `ID_part3`. Durch den relationalen Rückblick auf den vorherigen Haltepunkt derselben physischen Verbindung (`ID_part3 <`) wird ermittelt, ob die Verspätung am untersuchten Knotenpunkt originär entstanden ist oder als Primärverspätung aus dem vorgelagerten Schienennetz übertragen wurde.

[5]:
```python
# Definition des Fensters über die eindeutige Fahrt (Kombination aus part1 und␣
 ↪part2)
# Sortiert nach der Haltestellensequenz part3
fahrt_fenster = Window.partitionBy("ID_part1", "ID_part2").orderBy("ID_part3")

# Ermittlung der Ankunftsverspätung der physisch vorherigen existierenden␣
 ↪Haltestelle
df_spark_mit_historie = df_spark.withColumn(
    "vorherige_ankunfts_verspaetung_m",
    F.lag("arrival_delay_m", 1).over(fahrt_fenster)
).withColumn(
```

```
    "vorheriger_haltestellen_index",
    F.lag("ID_part3", 1).over(fahrt_fenster)
)

# Registrierung des erweiterten DataFrames als neue temporäre SQL-View
df_spark_mit_historie.createOrReplaceTempView("zug_daten_mit_historie")

print("Erweiterter Basis-DataFrame erfolgreich mit historischer Fensterfunktion␣
 ↪verknüpft.")
```

```
Erweiterter Basis-DataFrame erfolgreich mit historischer Fensterfunktion
verknüpft.
```

[6]:
```
# Definition der zeitlichen Grenzen in Sekunden (2 Min = 120s, 50 Min = 3000s)
MIN_TRANSFER_SEK = 120
MAX_TRANSFER_SEK = 3000

# SQL-Abfrage unter Verwendung von unix_timestamp() für TIMESTAMP_NTZ
geplante_anschluesse_query = f"""
SELECT
    -- Geografische und funktionale Schlüssel
    zubringer.eva_nr,
    zubringer.station,
    zubringer.state,

    -- Metriken des ankommenden Zubringer-Zuges
    zubringer.ID_part1 AS zubringer_fahrt_id,
    zubringer.line AS zubringer_linie,
    zubringer.arrival_plan AS zubringer_plan_ankunft,
    zubringer.arrival_change AS zubringer_tats_ankunft,
    zubringer.arrival_delay_m AS zubringer_ankunfts_verspaetung_m,
    zubringer.vorherige_ankunfts_verspaetung_m AS␣
 ↪zubringer_ursprung_verspaetung_m,
    zubringer.ID_part3 AS zubringer_aktueller_index,
    zubringer.vorheriger_haltestellen_index AS zubringer_vorheriger_index,

    -- Metriken des abfahrenden Anschluss-Zuges
    anschluss.ID_part1 AS anschluss_fahrt_id,
    anschluss.line AS anschluss_linie,
    anschluss.departure_plan AS anschluss_plan_abfahrt,
    anschluss.departure_change AS anschluss_tats_abfahrt,
    anschluss.departure_delay_m AS anschluss_abfahrts_verspaetung_m,

    -- Berechnete fahrplanmäßige Transferzeit in Minuten
    (UNIX_TIMESTAMP(anschluss.departure_plan) - UNIX_TIMESTAMP(zubringer.
 ↪arrival_plan)) / 60 AS geplante_transferzeit_m
```

```
FROM zug_daten_mit_historie AS zubringer
INNER JOIN zug_daten_mit_historie AS anschluss
    ON zubringer.eva_nr = anschluss.eva_nr
WHERE
    -- Ausschluss von Scheinkorrelationen (ein Zug kann nicht sein eigener Anschluss sein)
    zubringer.ID_part1 != anschluss.ID_part1

    -- Der Takt-Korridor für geplante Anschlüsse (2 bis 50 Minuten)
    AND (UNIX_TIMESTAMP(anschluss.departure_plan) - UNIX_TIMESTAMP(zubringer.arrival_plan)) >= {MIN_TRANSFER_SEK}
    AND (UNIX_TIMESTAMP(anschluss.departure_plan) - UNIX_TIMESTAMP(zubringer.arrival_plan)) <= {MAX_TRANSFER_SEK}
"""

# Ausführen der korrigierten Abfrage und Registrierung als primäre TempView
df_geplante_anschluesse = spark.sql(geplante_anschluesse_query)
df_geplante_anschluesse.createOrReplaceTempView("view_geplante_anschluesse")

# Cache aktivieren zur Performance-Sicherung der nachfolgenden kaskadierten Abfragen
df_geplante_anschluesse.persist(StorageLevel.MEMORY_AND_DISK)

# Auslösen der physikalischen Berechnung und Füllen des Caches (Lazy Evaluation beenden)
start_persist = time.perf_counter()
anzahl_geplante_anschluesse = df_geplante_anschluesse.count()
persist_zeit = time.perf_counter() - start_persist

print(f"--- Persistierungs-Status ---")
print(f"Anzahl generierter geplanter Anschlüsse (Baseline): {anzahl_geplante_anschluesse:,}")
print(f"Berechnungs- und Speicherzeit: {persist_zeit:.2f} Sekunden")

print("TempView 'view_geplante_anschluesse' erfolgreich zeitzonen-neutral erstellt und gecached.")
```

```
--- Persistierungs-Status ---
Anzahl generierter geplanter Anschlüsse (Baseline): 21,771,393
Berechnungs- und Speicherzeit: 43.45 Sekunden
TempView 'view_geplante_anschluesse' erfolgreich zeitzonen-neutral erstellt und gecached.
```

[7]:
```
# Sichern der Baseline-Kardinalität für die spätere Berechnungen
K_ANZAHL_GEPLANTE_ANSCHLUESSE = anzahl_geplante_anschluesse
```

1.6 Quantifizierung realisierter Anschlussverluste (Kollisionsanalyse)

Auf Basis der etablierten Baseline von über 21,7 Millionen geplanten Umstiegsverbindungen wird nachfolgend die Teilmenge der physisch verpassten Anschlüsse isoliert. Ein Anschlussverlust wird mathematisch als zeitliche Kollision definiert: Das Ereignis tritt ein, wenn die tatsächliche, verspätungsbehaftete Ankunftszeit des Zubringers (`zubringer_tats_ankunft`) chronologisch nach der fahrplanmäßig geplanten Abfahrtszeit des Anschlusszuges (`anschluss_plan_abfahrt`) liegt.

Da diese Selektion direkt auf der im Speicher partitionierten und persistierten Gesamtansicht operiert, ist eine hocheffiziente Filterung gewährleistet. Die Ergebnisse werden in einer neuen temporären Ansicht (`view_verpasste_anschluesse`) registriert, um das absolute Volumen der verpassten Anschlüsse im Gesamtnetz zu quantifizieren.

[8]:
```
# SQL-Abfrage zur Isolation der physisch kollidierten Anschlüsse
verpasste_anschluesse_query = """
SELECT
    *,
    -- Berechnung, um wie viele Minuten der Anschluss physisch verpasst wurde
    (UNIX_TIMESTAMP(zubringer_tats_ankunft) -␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt)) / 60 AS anschluss_verpasst_um_m
FROM view_geplante_anschluesse
WHERE
    -- Kollisions-Bedingung: Tatsächliche Ankunft des Zubringers NACH geplanter␣
 ↪Abfahrt des Anschlusses
    UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt)
"""

# Ausführen der Abfrage und Registrierung als sekundäre TempView
df_verpasste_anschluesse = spark.sql(verpasste_anschluesse_query)
df_verpasste_anschluesse.createOrReplaceTempView("view_verpasste_anschluesse")

# Da diese Tabelle für die spätere Kausalitätsprüfung gebraucht wird, wird sie␣
 ↪ebenfalls persistiert
df_verpasste_anschluesse.persist(StorageLevel.MEMORY_AND_DISK)

# Auslösen der Berechnung und Sichern der Anzahl als Konstante
start_calc_verpasst = time.perf_counter()
anzahl_verpasste_anschluesse = df_verpasste_anschluesse.count()
calc_verpasst_zeit = time.perf_counter() - start_calc_verpasst
# Speichern als globale Notebook-Konstante
K_ANZAHL_VERPASSTE_ANSCHLUESSE = anzahl_verpasste_anschluesse

print(f"--- Analyse der verpassten Anschlüsse abgeschlossen ---")
print(f"Anzahl physisch verpasster Anschlüsse: {K_ANZAHL_VERPASSTE_ANSCHLUESSE:
 ↪,}")
print(f"Berechnungszeit: {calc_verpasst_zeit:.2f} Sekunden")
```

```
# Berechnung der relativen Ausfallquote im Netz
quote_verpasst = (K_ANZAHL_VERPASSTE_ANSCHLUESSE /␣
 ↪K_ANZAHL_GEPLANTE_ANSCHLUESSE) * 100
print(f"Prozentualer Anteil verpasster Anschlüsse an allen geplanten␣
 ↪Verbindungen: {quote_verpasst:.2f}%")
```

```
--- Analyse der verpassten Anschlüsse abgeschlossen ---
Anzahl physisch verpasster Anschlüsse: 254,223
Berechnungszeit: 5.34 Sekunden
Prozentualer Anteil verpasster Anschlüsse an allen geplanten Verbindungen: 1.17%
```

1.7 Quantifizierung von Sekundärverspätungen durch betriebliche Warzeiten

Nach der Isolation der direkten Anschlussverluste wird im dritten Schritt der Kaskade das Phänomen der netzweiten Verspätungsübertragung (Sekundärverspätung) untersucht. Im operativen Bahnbetrieb werden Anschlusszüge häufig gezielt zurückgehalten (Wartezeit-Anordnung), um Umstiegsströme von verspäteten Zubringern aufzunehmen und einen physischen Anschlussverlust zu verhindern. Dies sichert zwar die Reisekette der Fahrgaste, nimmt jedoch die Verzögerung in den Folgezug und destabilisiert nachgelagerte Netzabschnitte mit.

Ein gewarteter Anschluss mit Sekundärverspätung wird über folgende mathematische Bedingungen definiert: 1. **Verspäteter Zubringer:** Das ankommende System weist eine reale Zeitabweichung auf (`zubringer_ankunfts_verspaetung_m > 0`). 2. **Erfolgreicher physischer Transfer:** Die tatsächliche Ankunft des Zubringers liegt chronologisch vor oder exakt auf der tatsächlichen Abfahrt des Anschlusses (`zubringer_tats_ankunft <= anschluss_tats_abfahrt`), sodass der Anschlusszug physisch erreicht wurde. 3. **Indizierte Folgeverspätung:** Der Anschlusszug fährt fahrplanwidrig verspätet ab (`anschluss_abfahrts_verspaetung_m > 0`).

Die Ergebnisse werden in der Ansicht `view_gewartete_anschluesse` aggregiert, um das Volumen dieses infektiösen Störungsfaktors im Schienennetz zu quantifizieren.

[9]:
```
# SQL-Abfrage zur Isolation der gewarteten Anschlüsse (Sekundärverspätungen)
gewartete_anschluesse_query = """
SELECT
    *,
    -- Berechnung der tatsächlichen verbliebenen Transferzeit für den Fahrgast␣
 ↪in Minuten
    (UNIX_TIMESTAMP(anschluss_tats_abfahrt) -␣
 ↪UNIX_TIMESTAMP(zubringer_tats_ankunft)) / 60 AS tatsaechliche_transferzeit_m
FROM view_geplante_anschluesse
WHERE
    -- Bedingung 1: Zubringer war definitiv verspätet
    zubringer_ankunfts_verspaetung_m > 0

    -- Bedingung 2: Der Anschlusszug fuhr ebenfalls verspätet ab
    AND anschluss_abfahrts_verspaetung_m > 0

    -- Bedingung 3: Der Anschluss wurde physisch noch erreicht␣
 ↪(Zubringer-Ankunft vor/bei Anschluss-Abfahrt)
```

```
    AND UNIX_TIMESTAMP(zubringer_tats_ankunft) <=␣
 ↪UNIX_TIMESTAMP(anschluss_tats_abfahrt)

    -- Bedingung 4: Kausales Wartezeit-Indiz (Tatsächliche Ankunft des␣
 ↪Zubringers lag NACH der geplanten Abfahrt des Anschlusses,
    -- was beweist, dass der Anschlusszug ohne das Warten pünktlich abgefahren␣
 ↪wäre)
    AND UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt)
"""

# Ausführen der Abfrage und Registrierung als TempView
df_gewartete_anschluesse = spark.sql(gewartete_anschluesse_query)
df_gewartete_anschluesse.createOrReplaceTempView("view_gewartete_anschluesse")

# Persistierung für die finale Auswertung und Kausalitätsprüfung
df_gewartete_anschluesse.persist(StorageLevel.MEMORY_AND_DISK)

# Auslösen der Berechnung und Sichern der Anzahl als Konstante
start_calc_gewartet = time.perf_counter()
anzahl_gewartete_anschluesse = df_gewartete_anschluesse.count()
calc_gewartet_zeit = time.perf_counter() - start_calc_gewartet
# Speichern als globale Notebook-Konstante
K_ANZAHL_GEWARTETE_ANSCHLUESSE = anzahl_gewartete_anschluesse

print(f"--- Analyse der gewarteten Anschlüsse (Sekundärverspätungen)␣
 ↪abgeschlossen ---")
print(f"Anzahl gewarteter Anschlüsse mit induzierter Folgeverspätung:␣
 ↪{K_ANZAHL_GEWARTETE_ANSCHLUESSE:,}")
print(f"Berechnungszeit: {calc_gewartet_zeit:.2f} Sekunden")

# Berechnung der relativen Quote am Gesamtnetz
quote_gewartet = (K_ANZAHL_GEWARTETE_ANSCHLUESSE /␣
 ↪K_ANZAHL_GEPLANTE_ANSCHLUESSE) * 100
print(f"Prozentualer Anteil übertragener Folgeverspätungen an allen␣
 ↪Verbindungen: {quote_gewartet:.2f}%")
```

```
--- Analyse der gewarteten Anschlüsse (Sekundärverspätungen) abgeschlossen ---
Anzahl gewarteter Anschlüsse mit induzierter Folgeverspätung: 90,539
Berechnungszeit: 4.73 Sekunden
Prozentualer Anteil übertragener Folgeverspätungen an allen Verbindungen: 0.42%
```

1.8 Analyse der historischen Kausalitätsketten im Schienennetz

Nach der Trennung und Quantifizierung der beiden primären Störungsfaktoren (254.223 verpasste Anschlüsse und 90.539 induzierte Sekundärverspätungen) wird im finalen Schritt der PySpark-Pipeline die Kausalität dieser Netzstörungen untersucht. Es wird mathematisch isoliert, ob der

jeweilige Zubringer-Zug die Netzstörung am aktuellen Netzknotenpunkt originär verursacht hat oder ob eine fortlaufende Verspätungsübertragung aus vorgelagerten Streckenabschnitten vorliegt.

Die Klassifizierung der Kausalität basiert auf dem zuvor über die partitionierte Fensterfunktion generierten historischen Attribut `zubringer_ursprung_verspaetung_m`: 1. **Originäre Netzknoten-Verspätung (Neu entstanden):** Der Zubringer-Zug wies an der unmittelbar vorangegangenen, im Datensatz erfassten Haltestelle eine Verspätung von null Minuten auf (oder operierte an seinem Startbahnhof) und hat die fahrplanreißende Zeitabweichung erst auf dem aktuellen Streckenabschnitt akkumuliert. 3. **Übertragene Netzknoten-Verspätung (Eingeschleppt):** Der Zubringer-Zug war bereits an der vorherigen Station mit einer Verspätung behaftet. Die Anschlussstörung am aktuellen Knotenpunkt ist somit das direkte Resultat einer durch das Netz geschleppten Primärverspätung.

Die statistische Aggregation dieser beiden Klassen liefert die empirische Basis, um den strukturellen Infektionsgrad des Schienennetzes im Rahmen der finalen Dokumentation zu bewerten.

[10]:
```
# SQL-Abfrage zur Zusammenführung aller gestörten Anschlüsse und deren␣
 ↪Kausalitäts-Klassifizierung
kausalitaet_query = """
WITH union_stoerungen AS (
    -- Gruppe 1: Verpasste Anschlüsse
    SELECT zubringer_fahrt_id, zubringer_ankunfts_verspaetung_m,␣
 ↪zubringer_ursprung_verspaetung_m, 'Verpasst' as stoerungstyp
    FROM view_verpasste_anschluesse

    UNION ALL

    -- Gruppe 2: Gewartete Anschlüsse
    SELECT zubringer_fahrt_id, zubringer_ankunfts_verspaetung_m,␣
 ↪zubringer_ursprung_verspaetung_m, 'Gewartet' as stoerungstyp
    FROM view_gewartete_anschluesse
)
SELECT
    stoerungstyp,
    COUNT(*) AS gesamt_faelle,
    -- Berechnung der übertragenen Fälle (Vorherige Station hatte bereits␣
 ↪Verspätung > 0)
    SUM(CASE WHEN zubringer_ursprung_verspaetung_m > 0 THEN 1 ELSE 0 END) AS␣
 ↪faelle_uebertragen,
    -- Berechnung der originären Fälle (Vorherige Station war pünktlich oder␣
 ↪None/Startbahnhof)
    SUM(CASE WHEN zubringer_ursprung_verspaetung_m = 0 OR␣
 ↪zubringer_ursprung_verspaetung_m IS NULL THEN 1 ELSE 0 END) AS␣
 ↪faelle_originaer
FROM union_stoerungen
GROUP BY stoerungstyp
"""
```

```
# Ausführen der finalen Kausalitätsabfrage
df_kausalitaet = spark.sql(kausalitaet_query)

print("--- Ergebnisse der historischen Kausalitätsanalyse ---")
df_kausalitaet.show()
```

```
--- Ergebnisse der historischen Kausalitätsanalyse ---
+------------+-------------+------------------+----------------+
|stoerungstyp|gesamt_faelle|faelle_uebertragen|faelle_originaer|
+------------+-------------+------------------+----------------+
|    Verpasst|       254223|            220450|           33773|
|    Gewartet|        90539|             79978|           10561|
+------------+-------------+------------------+----------------+
```

[11]:
```
# Zusammenführung der berechneten Metriken zu einem finalen Analyse-DataFrame␣
 ↪für den Export
# Aggregiert die Daten nach Bundesland und Bahnhof, um später im␣
 ↪Projekt-Notebook interaktive Filter zu ermöglichen
final_dashboard_export_query = """
SELECT
    eva_nr,
    station,
    state,
    COUNT(*) AS gesamt_geplante_anschluesse,
    SUM(CASE WHEN UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt) THEN 1 ELSE 0 END) AS␣
 ↪anzahl_verpasste_anschluesse,
    SUM(CASE WHEN zubringer_ankunfts_verspaetung_m > 0 AND␣
 ↪anschluss_abfahrts_verspaetung_m > 0
             AND UNIX_TIMESTAMP(zubringer_tats_ankunft) <=␣
 ↪UNIX_TIMESTAMP(anschluss_tats_abfahrt)
             AND UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt) THEN 1 ELSE 0 END) AS␣
 ↪anzahl_gewartete_anschluesse,
    SUM(CASE WHEN zubringer_ursprung_verspaetung_m > 0 AND␣
 ↪(UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt)) THEN 1 ELSE 0 END) AS␣
 ↪verpasst_wegen_vorverspaetung,
    SUM(CASE WHEN zubringer_ursprung_verspaetung_m > 0 AND␣
 ↪(zubringer_ankunfts_verspaetung_m > 0 AND anschluss_abfahrts_verspaetung_m >␣
 ↪0
             AND UNIX_TIMESTAMP(zubringer_tats_ankunft) <=␣
 ↪UNIX_TIMESTAMP(anschluss_tats_abfahrt)
             AND UNIX_TIMESTAMP(zubringer_tats_ankunft) >␣
 ↪UNIX_TIMESTAMP(anschluss_plan_abfahrt)) THEN 1 ELSE 0 END) AS␣
 ↪folgeverspaetung_durch_warten
```

```
FROM view_geplante_anschluesse
GROUP BY eva_nr, station, state
"""
```

[12]:
```
print("Starte finale Aggregation in PySpark...")
start_aggregation = time.perf_counter()

# 1. Ausführen der bewährten SQL-Abfrage aus der SparkSession
df_export_dashboard = spark.sql(final_dashboard_export_query)

print("Überführe hochverdichtete Ergebnisse in ein Pandas-DataFrame (toPandas)..
 ↪.")
# 2. Konvertierung des aggregierten Spark-DataFrames in ein Pandas-DataFrame
pdf_dashboard_export = df_export_dashboard.toPandas()

# 3. Ermittlung und Konvertierung des Zielpfads in ein relatives POSIX-Format
EXPORT_RELATIVE_PARQUET_STRING = get_project_relative_path(PYSPARK_RESULTS)
physischer_ziel_pfad = Path(EXPORT_RELATIVE_PARQUET_STRING)

# 4. Automatisierte Bereinigung blockierender Altlasten auf der Festplatte
if physischer_ziel_pfad.exists():
    if physischer_ziel_pfad.is_dir():
        print(f"Entferne blockierendes Altziel (Verzeichnis):␣
 ↪{EXPORT_RELATIVE_PARQUET_STRING}")
        shutil.rmtree(physischer_ziel_pfad)
    else:
        print(f"Entferne existierende Altdatei:␣
 ↪{EXPORT_RELATIVE_PARQUET_STRING}")
        physischer_ziel_pfad.unlink()

print(f"Sichere bereinigte Ergebnis-Basis als Parquet-Datei unter (relativ):␣
 ↪{EXPORT_RELATIVE_PARQUET_STRING}")

# 5. Verlustfreier nativer Export als einzelne Parquet-Datei via Pandas␣
 ↪(PyArrow)
pdf_dashboard_export.to_parquet(EXPORT_RELATIVE_PARQUET_STRING, index=False)

export_zeit = time.perf_counter() - start_aggregation

print("\n" + "="*70)
print("ERFOLG: Aggregierte Ergebnisse als einheitliche Parquet-Datei gesichert!
 ↪")
print(f"Dimensionen der Export-Tabelle: {pdf_dashboard_export.shape}")
print(f"Gesamte Verarbeitungs- und Exportzeit: {export_zeit:.2f} Seconds")
print("="*70)
```

```
Starte finale Aggregation in PySpark…
```

```
Überführe hochverdichtete Ergebnisse in ein Pandas-DataFrame (toPandas)…

C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:298:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()

Entferne existierende Altdatei: outputs/03_results.parquet
Sichere bereinigte Ergebnis-Basis als Parquet-Datei unter (relativ):
outputs/03_results.parquet

======================================================================
ERFOLG: Aggregierte Ergebnisse als einheitliche Parquet-Datei gesichert!
Dimensionen der Export-Tabelle: (1941, 8)
Gesamte Verarbeitungs- und Exportzeit: 4.40 Seconds
======================================================================
```

[13]:
```
print("Beende die aktive SparkSession und gebe Systemressourcen frei...")
spark.stop()
print("SparkSession erfolgreich geschlossen.")
```

```
Beende die aktive SparkSession und gebe Systemressourcen frei…
SparkSession erfolgreich geschlossen.
```

[ ]:
```

```